

經濟部所屬事業機構 108 年新進職員甄試試題

類別：資訊

科目：1. 資訊管理 2. 程式設計

一、某公司的關聯式資料庫包含下列表格，有底線者為主鍵：（22 分）

貨品(貨號，品名，單價，庫存數量，供應商編號，供應商名稱)

供應商(供應商編號，供應商名稱，供應商地址，供應商電話，聯絡人)

(一)請用 SQL 列出下列查詢：（14 分）

(1)所有供應商名稱開頭為「台」的供應商編號、供應商名稱、供應商地址、供應商電話及聯絡人。（3 分）

(2)所有庫存數量為「0」的貨品，其貨號、品名、單價、供應商名稱及聯絡人。（4 分）

(3)至少供應 2 種貨品之供應商名稱，及該供應商販售幾種貨品。（7 分）

(二)就上述資料庫，回答下列問題：（8 分）

(1)資料庫設計上有何問題？（2 分）

(2)資料庫運作有何缺點？（4 分）

(3)資料庫運作有何優點？（2 分）

擬答：

(一)

(1) SELECT 供應商編號, 供應商名稱, 供應商地址, 供應商電話, 聯絡人
FROM 供應商

WHERE 供應商名稱 LIKE “台%”

(2) SELECT 貨號, 品名, 單價, 供應商名稱, 聯絡人
FROM 貨品, 供應商

WHERE 貨品.供應商編號 = 供應商.供應商編號 AND 庫存數量=0

(3) SELECT 供應商名稱, COUNT(*)
FROM 貨品

GROUP BY 供應商名稱

HAVING COUNT(*)>1

(二)

(1)由於在貨品表格中有 FD 供應商編號→供應商名稱，由於供應商編號非候選鍵，供應商名稱非鍵屬性，故此表格不合 3NF，而可能產生更改混淆問題。

(2)本題資料庫設計由於不合 3NF，可能有下列更改混淆的運作缺點：

①輸入時必須等主鍵輸入才可進行：例如貨品表格中我們不能單單描述供應商編號的供應商名稱，一直要到確實有某個貨號輸入後才能將上列資料輸入。（因為實體整合規則）

②刪除時會把過多資訊刪掉：例如貨品表格我們將一個值組刪除時，我們同時也刪掉供應商編號對應的供應商名稱資訊。

③修改時需要一起修改許多相關值組：例如貨品表格中我們將一個供應商編號對應的供應商名稱修改時，則需要將其其他的值組一起修改，否則資料會不一致。

(3)本題資料庫運作由於有重複存放部分資料，部分查詢的速度會加快。

二、簡答題：（22 分）

(一)請說明何謂單元測試（Unit test）？（4 分）

(二)請說明何謂同名異式（Polymorphism）及封裝（Encapsulation）？（6 分）

(三)某公司的資訊系統，使用者登入時約 30 秒才有回應，請說明此現象和資訊安全的哪一個特性最相關？（2 分）

(四)請說明死結 (Dead Lock) 發生之條件? (4 分)

(五)github.com 最近被併購, git 是一種開源軟體, 請說明git 最重要之功能為何? (2 分)

(六)請說明何謂超荷 (Overload) 及覆寫 (Override)? (4 分)

擬答:

(一)單元測試 (Unit Testing) 是針對程式模組 (軟體設計的最小單位) 來進行正確性檢驗的測試工作。程式單元是應用的最小可測試部件。在程序化編程中, 一個單元就是單個程式、函式、過程等; 對於物件導向程式設計, 最小單元就是方法, 包括基礎類別 (超類)、抽象類、或者衍生類別 (子類) 中的方法。

(二)多型 (polymorphism) 指為不同資料類型的實體提供統一的介面。電腦程式執行時, 相同的訊息可能會送給多個不同的類別之物件, 而系統可依據物件所屬類別, 引發對應類別的方法, 而有不同的行為。簡單來說, 所謂多型意指相同的訊息給予不同的物件會引發不同的動作。封裝 (Encapsulation) 是指, 一種將抽象性函式介面的實作細節部份包裝、隱藏起來的方法。同時, 它也是一種防止外界呼叫端, 去存取物件內部實作細節的手段, 這個手段是由程式語言本身來提供的。

(三)此與可用性 (Availability) 有關, 可用性是指系統, 子系統, 或者設備在開始一項任務時處在指定的可操作或可提交狀態的程度, 這項任務什麼時候被用到是未知的, 也就是隨機的。簡單的說, 可用性就是一個系統處在可工作狀態的時間的比例。

(四)死結的四個條件是:

1. 禁止搶占 (no preemption) - 系統資源不能被強制從一個行程中退出
2. 持有和等待 (hold and wait) - 一個行程可以在等待時持有系統資源
3. 互斥 (mutual exclusion) - 只有一個行程能持有一個資源
4. 循環等待 (circular waiting) - 一系列行程互相持有其他行程所需要的資源

(五)Git 是一種版本控制系統, 遇到需要存好幾種版本的備份, 通常我們會在電腦新增一個資料夾, 並將備份後的報告丟進去, 然後替每一個報告取一個版號或是日期戳記, 以便有時候我們可能會需要以前的版本來還原。

(六)超荷 (Overload) 是數項名稱相同但輸入輸出類型或個數不同的子程式, 它可以簡單地稱為一個單獨功能可以執行多項任務的能力。覆寫 (Override) 則是物件導向語言的一種功能, 允許次類別提供其超類別或父類別之一已經提供的方法的特定實現。次類別中的實現通過提供與父類別中的方法具有相同名稱, 相同參數以及相同返回類型的方法來覆蓋 (替換) 父類別中的實現。

三、巨量資料題組: (6 分)

(一)若 $P(A) = 0.5$, $P(B) = 0.2$, $P(A|B) = 0.3$, $P(A \cup B) = ?$ (4 分)

(二)請說明何謂過度訓練(Overfitting)? (2 分)

擬答:

(一) $P(A|B) = P(A \cap B)/P(B)$, 因此 $P(A \cap B) = 0.3 * 0.2 = 0.06$, $P(A \cup B) = P(A) + P(B) - P(A \cap B) = 0.5 + 0.2 - 0.06 = 0.64$ 。

(二)是指過於緊密或精確地匹配特定資料集的結果, 以致於無法擬合其他資料或預測未來的觀察結果。在過適的過程中, 當預測訓練範例結果的表現增加時, 應用在未知資料的表現則變得更差。在統計和機器學習中, 為了避免過適現象, 須要使用額外的技巧 (如交叉驗證、提早停止、貝斯資訊量準則、赤池資訊量準則或模型比較), 以指出何時會有更多訓練而沒有導致更好的一般化。

四、有一 Web 程式系統, 由使用者輸入 ID 及地址, 並由後端接收處理程式進行資料庫相關處理, 其前端畫面輸入欄位、對應之 Html 碼及後端接收處理程式如下, 試回答下列問題: (14 分)

前端畫面輸入欄位：

請輸入 ID：

請輸入地址：

確定

取消

對應之輸入欄位 Html 碼：

<input type="text" name="id">

<input type="text" name="address">

後端接收處理程式及註解(雙斜線//後為註解)：

<%

.....//以上省略

\$id = Request(" id "); //取得輸入 ID 欄位資料

\$address = Request(" address "); //取得輸入地址欄位資料

\$sqlstring = " Select * from idtable where id = '"+\$id+"' " ; //id 是否存在於 idtable 資料表

\$result = sql_query(\$sqlstring); //進行資料庫查詢及結果

.....//以下省略

%>

(一)請問上述設計會產生何種資料庫資安風險？(2 分)

(二)請列舉上述資安風險對系統及資料所產生之危害。(4 分)

(三)請在不變動系統任何設定及前端輸入下，就你所知道之任一程式語言(含虛擬碼)，在後端接收處理程式新增字串處理函數 checkdata(xxxx)，傳入參數為使用者輸入之資料，經處理後傳回無風險之資料(除宣告函數外，每行程式碼皆須註解處理內容，未加註解者不予計分)。(8 分)

擬答：

(一)可能會有被 SQL injection 攻擊的風險。

(二)產生之危害

1. 資料表中的資料外洩，例如企業及個人機密資料，帳戶資料，密碼等。企業網站首頁被竄改，門面盡失。
2. 資料結構被駭客探知，得以做進一步攻擊（例如 SELECT * FROM sys.tables）。破壞硬碟資料，癱瘓全系統（例如 xp_cmdshell "FORMAT C:"）。
3. 資料庫伺服器被攻擊，系統管理員帳戶被竄改（例如 ALTER LOGIN sa WITH PASSWORD='xxxxxx'）。取得系統較高權限後，有可能得以在網頁加入惡意連結、惡意代碼以及 Phishing 等。取得系統最高權限後，可針對企業內部的任一管理系統做大規模破壞，甚至讓其企業倒閉。
4. 經由資料庫伺服器提供的作業系統支援，讓駭客得以修改或控制作業系統（例如 xp_cmdshell "net stop iisadmin"可停止伺服器的 IIS 服務）。駭客經由上傳 php 簡單的指令至對方之主機內，PHP 之強大系統命令，可以讓駭客進行全面控制系統(例如:php 一句話木馬)。

(三)如果是用 ASP.NET 寫，可以使用 Parameters 改寫

```
01protected void Verify_Click(object sender, EventArgs e)
02{
03    string connectionString = @"myConnectionString";
04    var id = this.id.Text;
05    var address = this.address.Text;
06
```

```
07 int count;
08 using (SqlConnection cn = new SqlConnection(connectionString))
09 {
10     cn.Open();
11     string sqlStatement = @"Select Count(1) From SomeTable Where ID= @id AND address=
12     @address";
13     SqlCommand sqlCommand = new SqlCommand(sqlStatement, cn);
14     ////定義 parameter 型別
15     sqlCommand.Parameters.Add("@id", SqlDbType.VarBinary);
16     sqlCommand.Parameters["@id"].Value = id;
17
18     ////讓 ADO.NET 自行判斷型別轉換
19     sqlCommand.Parameters.AddWithValue("@address", address);
20
21     count = (int)sqlCommand.ExecuteScalar();
22 }
23
24 this.Result.Text = (count > 0) ? "Pass" : "帳號或密碼錯誤";
25}
```

五、請就二元搜尋樹回答下列問題：（20 分）

(一)請說明欲建立二元搜尋樹，必須滿足哪些條件？（6 分）

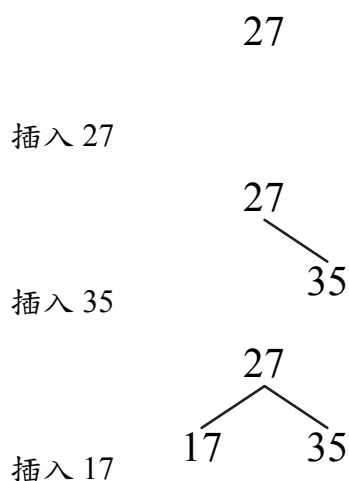
(二)數列 27、35、17、33、20、3、38，試以第 1 個數字為根，寫出其二元搜尋樹及建立的步驟。（10 分）

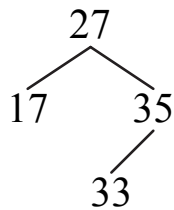
(三)在上述二元搜尋樹中，若欲刪除元素 27，請寫出 2 種做法。（4 分）

擬答：

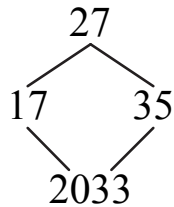
- (一)1. 若任意節點的左子樹不空，則左子樹上所有節點的值均小於它的根節點的值；
2. 若任意節點的右子樹不空，則右子樹上所有節點的值均大於它的根節點的值；
3. 任意節點的左、右子樹也分別為二元搜尋樹；
4. 沒有鍵值相等的節點。

(二)過程如下

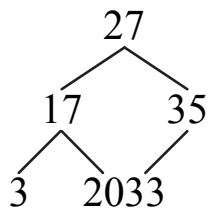




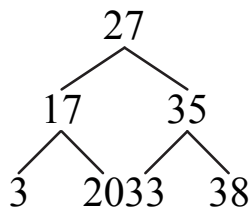
插入 33



插入 20

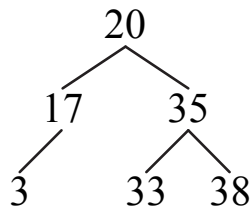


插入 3

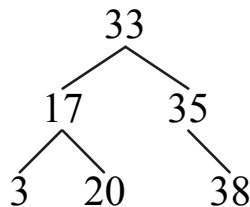


插入 38

(三) 1. 刪除 27 後，以左子樹最大值 20 取代，結果如下：



2. 刪除 27 後，以右子樹最小值 33 取代，結果如下：



六、在常見的程式設計語言中，變數常區分為全域變數（Global Variable）與區域變數（Local Variable），並在某些情況下使用靜態變數（Static Variable），試回答下列問題：（16 分）

(一) 何謂靜態變數與區域變數？並說明兩者的特性、差別及個別的生命週期。（6 分）

(二) 在一般程式設計中，若區域變數與全域變數同名，試問何者優先使用？（2 分）

(三)有一打彈珠機函數 balls，接收引數 input1、input2 分別代表輸入值與輸出倍數，其功能要求：80 %機率失敗，回傳值為 0；20 %機率成功，回傳值 = 輸入值*輸出倍數，宣告一變數 count 初始值 1,000，並記錄每次呼叫後之剩餘值。請在下列底線部分填入適當程式碼：（每項2分，共8分）

```
int balls ( int input1, int input2 ) {    // input1：輸入值；input2：輸出倍數
    _____ (1) _____
    int output = 0 ;
    if ( rand() >= _____ (2) _____ )    // rand()為亂數函數，0 <= rand() < 1
    {
        _____ (3) _____
    }
    count = _____ (4) _____
    return output ;
}
```

擬答：

(一)靜態變數在程式執行開始之前，就繫合到儲存單元上，且在程式執行結束之前，一直繫合在相同的儲存單元上。堆疊動態變數在宣告確立之後才產生儲存繫合，而它們的類型是靜態繫結的，其儲存空間是執行時從堆疊中分配。

種 類	靜 態 變 數	區 域 變 數
內 容	就是宣告成全域變數或以static 修飾字宣告的變數，在程式載入階段配置記憶體在執行記憶體的Data 或BSS 區段	這是宣告在函數中的一般變數或參數，在函數被執行建立活動紀錄 (Activation record,又稱為Stack Frame)時才會配置到區域變數或參數的記憶體區塊中，會隨著函數動態建立及釋放。
優 點	(1)執行效能佳：記憶體直接定址，不需要額外運算就能存取內容。 (2)可支援歷史紀錄：不隨函數釋放而清除內容，故可記錄函數執行歷史。 (3)提供函數與類別間溝通：不受物件釋放的影响，可做資料分享與傳遞訊息。	(1)支援遞迴程式：此類變數支援函數動態配置，可支援無法預測執行次數的遞迴程式。 (2)提高記憶體運用彈性：函數執行完後空間被自動釋放，可供其他函數配置使用。
缺 點	(1)記憶體運用缺乏彈性：配置固定記憶體空間，不能隨意增加與釋放。 (2)不支援遞迴程式：由於不知道執行次數，因此不能配置遞迴程式所需的記憶體。	(1)效能較差：執行時必須花費額外的系統處理時間配置、釋放記憶體。 (2)不支援歷史紀錄：無法儲存函數執行歷史，無法支援需要記錄過程的程式設計。
生 命 週 期	從程式載入階段配置記憶體直到整個程式結束釋放執行記憶體。	由配置函數活動紀錄時開始，到函數結束活動紀錄釋放時一起釋放。

(二)在局部變數的可見領域內，優先使用局部變數，若無可用的局部變數，則選用全域變數。

(三)(1) `static int count = 1000;`

(2) `0.8`

(3) `output = input1*input2;`

(4) `count - 1;`

公
職
王